

UNITÉ 4 · SEMAINES 5–6

Processus et mémoire

📅 Semaines 5–6 · dates à confirmer </> Laboratoire TD3 (corrigé + défi IA) ~ 7 animations

LABORATOIRE TD3 · CORRIGÉ

Tampons, fork, exec et wait

Ce que fait la bibliothèque C au-dessus des appels système, ce qui est copié quand un processus se duplique, et comment un programme en lance un autre et apprend comment il s'est terminé.

✓ **Devoir : rapport du TP**

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|---|---|---|
| 1 | Appel système ou fonction de bibliothèque ? | 4 | launch : rediriger et exécuter un autre programme |
| 2 | Les trois modes de mise en tampon de stdio | 5 | Exploration : la mémoire virtuelle et la vie d'un processus (nouveau) |
| 3 | fork et le tampon dupliqué | | |

TD3 · 1

● Corrigé

Appel système ou fonction de bibliothèque ?

Énoncé. Classez: printf, fopen, fclose, fread, open, close, lseek, rewind, write, exit, _exit. Servez-vous de leurs prototypes et de leur section du manuel.

Solution



APPELS SYSTÈME (SECTION 2)

`open`, `close`, `lseek`, `write`,
`_exit`

FONCTIONS DE LA BIBLIOTHÈQUE C (SECTION 3)

`printf`, `fopen`, `fclose`, `fread`, `rewind`,
`exit`

L'indice est dans les types: les appels travaillent avec des descripteurs (`int fd`); les fonctions de stdio, avec `FILE *`, une structure de la bibliothèque qui contient un descripteur **et un tampon**. `exit` vide ces tampons et exécute les fonctions `atexit` avant d'appeler `_exit`, qui termine le processus immédiatement.

TD3 · 2

● Corrigé · sorties réelles

Les trois modes de mise en tampon de stdio

Énoncé. `buffer.c` écrit «Hello», dort une seconde et écrit «World!». Avec des macros du préprocesseur, choisissez pour chaque chaîne `fprintf` ou `write`, `stdout` ou `stderr`, et à la fin `exit` ou `_exit`. Lancez chaque version dans le terminal et à travers un tube (`2>&1 | cat -u`) et identifiez les trois modes de mise en tampon.

Solution

C tds/td3/buffer.c

```
/*
 * TD3 - exercise 2: the three buffering modes of the C library.
 * One source, several behaviours chosen at compile time (see the Makefile
 * variants):
 *   STRING1, STRING2      the two strings (default "Hello " and
 *                           "World!\n")
 *   USE_FPRINTF1/2        write string 1/2 with fprintf (buffered) instead
 *                           of write (not buffered)
 *   USE_STDERR1/2         write string 1/2 on stderr instead of stdout
 *   USE__EXIT              end with _exit() (no buffer flush) instead of
 *                           exit()
 * Run each variant twice: ./buffer_N (terminal) and ./buffer_N 2>&1 |
 *                           cat -u (pipe)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

#include <unistd.h>

#include "utils.h"

#ifndef STRING1
#define STRING1 "Hello "
#endif
#ifndef STRING2
#define STRING2 "World!\n"
#endif

#ifdef USE_STDERR1
#define STREAM1 stderr
#define FD1 STDERR_FILENO
#else
#define STREAM1 stdout
#define FD1 STDOUT_FILENO
#endif

#ifdef USE_STDERR2
#define STREAM2 stderr
#define FD2 STDERR_FILENO
#else
#define STREAM2 stdout
#define FD2 STDOUT_FILENO
#endif

int main(void)
{
#ifdef USE_FPRINTF1
    fputs(STRING1, STREAM1);           /* goes to the
    stdio buffer */
#else
    exit_if(write(FD1, STRING1, strlen(STRING1)) == -1, "write"); /*
    straight to the kernel */
#endif

    sleep(1);

#ifdef USE_FPRINTF2
    fputs(STRING2, STREAM2);
#else
    exit_if(write(FD2, STRING2, strlen(STRING2)) == -1, "write");
#endif
}

```

```
#ifndef USE__EXIT
    _exit(EXIT_SUCCESS);          /* system call: the stdio buffers are NOT
flushed */
#else
    exit(EXIT_SUCCESS);          /* library function: flushes the stdio
buffers, then _exit */
#endif
}
```

VERSION (MAKEFILE)	TERMINAL	TUBE	POURQUOI
buffer_1 : fprintf(stdout) + write	World! · Hello	World! · Hello	«Hello» n'a pas de <code>\n</code> : même la mise en tampon par ligne ne le vide pas; il ne sort qu'à l' <code>exit</code> .
buffer_2 : write + fprintf(stderr)	Hello World!	Hello World!	<code>stderr</code> n'a pas de tampon: chaque <code>fprintf</code> est un <code>write</code> immédiat.
buffer_3 : fprintf(stdout) + fprintf(stderr)	World! · Hello	World! · Hello	Le flux d'erreur sort tout de suite; la sortie standard attend.
buffer_4 : comme 3, avec "Hello\n"	Hello · World!	World! · Hello	Dans un terminal, <code>stdout</code> est en tampon <i>par ligne</i> (le <code>\n</code> le vide); dans un tube, il est en tampon <i>complet</i> (4 Kio), donc il attend l' <code>exit</code> .
buffer_5 : fprintf(stdout) + <code>_exit</code>	World!	World!	<code>_exit</code> ne vide pas le tampon: «Hello» est perdu.

Sortie réelle

```
$ ./buffer_4          # en una terminal
Hello
World!
```

Sortie réelle

```
$ ./buffer_4 2>&1 | cat -u
World!
Hello
```

Sortie réelle

```
$ ./buffer_5 2>&1 | cat -u
World!
```

Les trois modes: **sans tampon** (`stderr`), **tampon par ligne** (`stdout` vers un terminal) et **tampon complet** (`stdout` vers un fichier ou un tube). `write` n'a jamais de tampon visible pour le programme: ce qu'il remet, le noyau l'a déjà.

[Animation A4.6 · Tampons de stdio →](#)

TD3 · 3

● Corrigé · testé

fork et le tampon dupliqué

Énoncé. Un programme affiche son pid **sans saut de ligne** avec `printf` et se duplique avec `fork`; l'enfant affiche son pid et celui de son parent. Le parent ne doit rien afficher après le `fork`. Que se passe-t-il? Proposez trois solutions.

Solution

C tds/td3/fork_pid.c

```
/*
 * TD3 - exercise 3: print the pid WITHOUT a newline, fork, and let only the
 * child print.
 * SOLUTION=0 the bug: "My pid is ..." appears twice (the unflushed stdio
 * buffer is copied by fork)
 * SOLUTION=1 fflush(stdout) before fork()
 * SOLUTION=2 no stdio buffer for stdout: setvbuf(stdout, NULL, _IONBF, 0)
 * SOLUTION=3 the parent leaves with _exit(): its copy of the buffer is
 * never flushed,
 * the child's copy is printed together with the child's line
 * Make builds the four variants: fork_pid_0 ... fork_pid_3.
```

```

    */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#ifndef SOLUTION
#define SOLUTION 1
#endif

int main(void)
{
    #if SOLUTION == 2
        exit_if(setvbuf(stdout, NULL, _IONBF, 0) != 0, "setvbuf");
    #endif

    printf("My pid is %d / ", getpid());    /* no '\n': stays in the buffer
    */
    #if SOLUTION == 1
        fflush(stdout);
    #endif

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                          /* child */
        printf("child %d, parent %d\n", getpid(), getppid());
        exit(EXIT_SUCCESS);                  /* exit flushes the child's
    buffer */
    }

    /* Parent: prints nothing after fork, but must wait for its child. */
    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    #if SOLUTION == 3
        _exit(EXIT_SUCCESS);                  /* do not flush the parent's
    copy */
    #else
        return EXIT_SUCCESS;                  /* returning from main =
    exit(): flushes */
    #endif
}

```

Sortie réelle

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Sortie réelle

```
$ ./fork_pid_1 | cat
My pid is 356458 / child 356460, parent 356458
```

Sortie réelle

```
$ ./fork_pid_3 | cat
My pid is 356482 / child 356484, parent 356482
```

Le problème. «My pid is ... / » est resté dans le tampon de `stdout`, qui vit dans la mémoire du processus. `fork` copie toute la mémoire, tampon compris: parent et enfant ont le même texte en attente et le vident tous les deux en se terminant, donc il apparaît deux fois alors que le parent «n'affiche rien». **Solutions:** (1) `fflush(stdout)` avant `fork`; (2) supprimer le tampon avec `setvbuf(..., _IONBF, 0)`; (3) faire terminer le parent avec `_exit`, qui ne vide pas sa copie: le texte sort une fois, depuis l'enfant. La 1 est la plus claire; la règle générale est de *vider les tampons avant* `fork`.

TD3 · 4

● Corrigé · testé

Launch : rediriger et exécuter un autre programme

Énoncé. `./launch sortie commande arg1 arg2...` doit se comporter comme `commande arg1 arg2... > sortie`: il écrit la ligne de commande dans `sortie`, crée un processus enfant qui redirige sa sortie standard avec `dup2` et exécute la commande avec `execvp`, et le parent indique comment elle s'est terminée (`wait(2)`). Testez-le avec un programme qui provoque une erreur de segmentation.

Solution

```
C tds/td3/launch.c

/*
 * TD3 - exercise 4: ./launch output command [args...] behaves like command
```

```

args... > output
* 1. writes the command line into `output`
* 2. forks; the child redirects its stdout to `output` with dup2 and runs
the command (execvp)
* 3. the parent waits and reports how the command ended (exit status or
signal)
*/
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s output command [args...]\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* O_TRUNC: like "> output", previous content is discarded. 0644 = rw-r--
r-- */
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, argv[1]);

    for (int i = 2; i < argc; i++) {
        exit_if(write_all(fd, argv[i], strlen(argv[i])) == -1, "write");
        exit_if(write_all(fd, i < argc - 1 ? " " : "\n", 1) == -1, "write");
    }

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {
        /* child */
        exit_if(dup2(fd, STDOUT_FILENO) == -1, "dup2");
        exit_if(close(fd) == -1, "close");
        /* 1 now points to the
file */
        execvp(argv[2], argv + 2);
        /* only returns on
error */
        perror(argv[2]);
        _exit(127);
        /* like the shell:
command not found */
    }
  
```

```

    }

    exit_if(close(fd) == -1, "close");           /* the parent does not
    need it */

    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    if (WIFEXITED(status)) {
        fprintf(stderr, "launch: %s exited with status %d\n", argv[2],
        WEXITSTATUS(status));
        return WEXITSTATUS(status);
    }
    if (WIFSIGNALED(status)) {
        fprintf(stderr, "launch: %s killed by signal %d (%s)%s\n", argv[2],
        WTERMSIG(status),
            signame(WTERMSIG(status)), WCOREDUMP(status) ? ", core
        dumped" : "");
        return 128 + WTERMSIG(status);         /* shell convention */
    }
    return EXIT_FAILURE;
}

```

C tds/td3/segfault.c

```
/*
 * TD3 - exercise 4: a program that dies with SIGSEGV, to test the error path
 of launch.
 * Address 0 is never mapped: the MMU raises a page fault, the kernel finds
 no mapping and
 * sends SIGSEGV to the process.
 */
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static void gen_segfault(void)
{
    volatile int *p = NULL;    /* volatile: the compiler must really do the
store */
    *p = 42;
}

int main(void)
{
    printf("about to write at address 0...\n");
    fflush(stdout);
    gen_segfault();
    return EXIT_SUCCESS;      /* never reached */
}
```

Sortie réelle

```
$ ./launch out.txt ls -1 demo; cat out.txt
launch: ls exited with status 0
ls -1 demo
a.txt
b.txt
```

Sortie réelle

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

Sortie réelle

```
$ ./launch out.txt nocommand
nocommand: No such file or directory
launch: nocommand exited with status 127
[exit status 127]
```

- `execvp` remplace le programme du processus *en conservant ses descripteurs* : c'est pourquoi la redirection se fait dans l'enfant, entre `fork` et `exec`. C'est ainsi que fonctionne le shell.
- `execvp` ne revient que s'il échoue : la ligne suivante est toujours une erreur (127 = «introuvable», convention du shell). On utilise `_exit` pour que l'enfant ne vide pas les tampons copiés du parent.
- `WIFEXITED` / `WEXITSTATUS` et `WIFSIGNALED` / `WTERMSIG` décodent le statut. 139 = 128 + 11 (`SIGSEGV`), la même convention que le shell dans `$?`.

⚠ Corrigée

La version héritée (`redirection.c`) ouvrait la sortie sans `O_TRUNC` (des restes d'exécutions précédentes plus longues subsistaient) et ignorait le résultat de `write` ; l'énoncé l'appelait `lance`.

TD3 · 5

● Nouveau · corrigé · testé

Exploration: la mémoire virtuelle et la vie d'un processus

Énoncé. Quatre petits programmes pour *mesurer* ce que racontent les diapositives 56 à 82: (1) `zones` affiche l'adresse du code, des données, du tas, d'un grand `malloc`, de `printf` et de la pile, et la région de `/proc/self/maps` qui contient chacune; lancez-le plusieurs fois. (2) `lazy` réserve 1 Gio et compte la mémoire résidente et les défauts de page avant et après en avoir touché la moitié. (3) `cow` remplit 256 Mio, fait un fork et compte les défauts de l'enfant en lecture et en écriture. (4) `lifecycle` montre un zombie avec `ps` avant et après `waitpid`, et un orphelin adopté.

Solution et sorties réelles



C tds/td3/zones.c

```

/*
 * TD3 - exploration 1: the memory zones of a process (original slide 62,
 * updated).
 * Prints the address of the code, the data, the heap, a large malloc, a
 * library function and the stack,
 * and the region of /proc/self/maps that contains each one. Run it twice:
 * with ASLR (address space
 * layout randomization) the addresses change on every run, but the order of
 * the zones does not.
 */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int global_array[10];           /* uninitialized global: .bss */
static int initialized = 42;    /* initialized global: .data */

/* Finds the line of /proc/self/maps whose range contains addr and prints its
 * permissions and name. */
static void show(const char *what, const void *addr)
{
    FILE *maps = fopen("/proc/self/maps", "r");
    exit_if(maps == NULL, "/proc/self/maps");

    char line[512];
    uintptr_t a = (uintptr_t)addr;
    while (fgets(line, sizeof line, maps) != NULL) {
        unsigned long start, end;
        char perms[5], path[256] = "";
        if (sscanf(line, "%lx-%lx %4s %*s %*s %*s %255[^\n]", &start, &end,
perms, path) < 3)
            continue;
        if (a ≥ start && a < end) {
            char *name = path;
            while (*name == ' ')
                name++;
            printf("%-24s %#14lx  %s  %s\n", what, (unsigned long)a, perms,
*name ? name : "(anonymous)");
            break;
        }
    }
}

```

```

    }
}

exit_if(fclose(maps) == EOF, "fclose");
}

int main(void)
{
    int local = 0;
    void *small = malloc(10);
    void *big = malloc(64 << 20);          /* above M_MMAP_THRESHOLD (128 KiB):
glibc uses mmap */
    exit_if(small == NULL || big == NULL, "malloc");

    printf("%-24s %14s  %s  %s\n", "zone", "address", "perm", "region");
    show("code   (main)", (const void *)main);
    show("data   (initialized)", &initialized);
    show("bss    (global_array)", global_array);
    show("heap   (malloc 10 B)", small);
    show("heap   (end: sbrk(0))", (char *)sbrk(0) - 1);
    show("mmap   (malloc 64 MiB)", big);
    show("libc   (printf)", (const void *)printf);
    show("stack  (local)", &local);

    free(big);
    free(small);
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./zones
zone           address      perm  region
code  (main)    0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data  (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss   (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap  (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap  (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap  (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc  (printf)      0x71b345064080  r-xp  /usr/lib/x86_64-linux-
gnu/libc.so.6
stack (local)      0x7fff6c1e68b4  rw-p  [stack]

```

Sortie réelle

```
$ for i in 1 2 3; do ./zones | grep main; done
code    (main)          0x57a1f784a350    r-xp  ~/fso/tds/bin/td3/zones
code    (main)          0x5d648e064350    r-xp  ~/fso/tds/bin/td3/zones
code    (main)          0x56e46bc21350    r-xp  ~/fso/tds/bin/td3/zones
```

L'ordre des zones reste le même (code, données et tas en bas; bibliothèques et `mmap` au-dessus; la pile tout en haut), mais la base change à chaque exécution: **ASLR**. Le `malloc` de 64 Mio n'est pas dans le tas: la glibc utilise `mmap` au-delà de 128 Kio.

C tds/td3/lazy.c

```
/*
 * TD3 - exploration 2: lazy allocation (original slides 59 and 61).
 * malloc of a large block returns at once and costs almost no physical
 * memory: the kernel only
 * reserves virtual addresses. Each page gets a physical frame the first time
 * it is touched, through a
 * (minor) page fault. The program measures the resident memory (VmRSS) and
 * the page faults
 * (getrusage) before and after touching half of the block.
 * ./lazy [MiB]          default: 1024 MiB
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

/* Resident set size in KiB, from /proc/self/status. */
static long rss_kib(void)
{
    FILE *f = fopen("/proc/self/status", "r");
    exit_if(f == NULL, "/proc/self/status");
```

```

char line[256];
long kib = -1;
while (fgets(line, sizeof line, f) != NULL)
    if (sscanf(line, "VmRSS: %ld kB", &kib) == 1)
        break;
exit_if(fclose(f) == EOF, "fclose");
return kib;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void report(const char *step, long f0)
{
    printf("%-34s VmRSS = %8ld KiB    page faults so far = %ld\n", step,
rss_kib(), minor_faults() - f0);
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 1024;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE);
    long f0 = minor_faults();

    report("start", f0);
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");
    printf("malloc(%zu MiB) returned %p\n", mib, (void *)p);
    report("after malloc", f0);

    long before = minor_faults();
    double t0 = now();
    for (size_t i = 0; i < size / 2; i += page)    /* touch one byte of
every page of the first half */
        p[i] = 1;
    double t1 = now();
    long faults = minor_faults() - before;
    report("after touching half of the pages", f0);
    printf("pages touched: %zu (%zu bytes each)    page faults: %ld    %.0f ns
per fault\n",
        size / 2 / page, page, faults, faults ? (t1 - t0) * 1e9 / faults :

```

```
0.0);

    free(p);
    report("after free", f0);
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./lazy 1024
start                                VmRSS =      1756 KiB   page faults so far
= 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =      1952 KiB   page faults so far
= 8
after touching half of the pages     VmRSS =    526300 KiB   page faults so far
= 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073   1968 ns per
fault
after free                            VmRSS =      2012 KiB   page faults so far
= 131081
```

`malloc(1 GiB)` ne change pas la mémoire résidente: il ne fait que réserver des adresses. Chaque page touchée pour la première fois provoque un défaut de page où le noyau alloue et met à zéro un cadre: autant de défauts que de pages.

C tds/td3/cow.c

```
/*
 * TD3 - exploration 3: copy-on-write after fork (original slide 74).
 * The parent fills a block, so all its pages are resident. After fork,
 * parent and child share the
 * physical pages, marked read-only. The child reads the whole block (no
 * copy: no page faults) and then
 * writes to a quarter of the pages: each first write causes a page fault in
 * which the kernel copies
 * that single page. The page faults are counted with getrusage.
 * ./cow [MiB]          default: 256 MiB
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <sys/wait.h>
#include <time.h>
```

```

#include <unistd.h>

#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 256;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE), pages =
size / page;
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");
    memset(p, 'x', size);                                /* every page is
now resident */
    printf("parent: %zu MiB filled (%zu pages of %zu bytes)\n", mib, pages,
page);
    fflush(stdout);                                       /* do not
duplicate the stdio buffer */

    double t0 = now();
    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        printf("fork took %.2f ms (only the page tables are copied)\n",
(now() - t0) * 1e3);

        long f = minor_faults();
        unsigned long sum = 0;
        for (size_t i = 0; i < size; i += page)
            sum += (unsigned char)p[i];
        printf("child reads %zu pages: %ld page faults (shared, nothing
copied; sum %lu)\n", pages, minor_faults() - f, sum);
    }
}

```

```

    f = minor_faults();
    double t1 = now();
    for (size_t i = 0; i < size / 4; i += page)
        p[i] = 'y';
    double t2 = now();
    long faults = minor_faults() - f;
    printf("child writes %zu pages: %ld page faults (one copy per page),
%.0f ns per copy\n",
        pages / 4, faults, faults ? (t2 - t1) * 1e9 / faults : 0.0);
    free(p);
    exit(EXIT_SUCCESS);
}

int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
printf("parent: the child exited with status %d; my copy still says
%c'\n", WEXITSTATUS(status), p[0]);
free(p);
return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per
copy
parent: the child exited with status 0; my copy still says 'x'

```

Après `fork`, lire ne copie rien (0 défaut); écrire provoque exactement un défaut — une copie — par page écrite. C'est pourquoi un fork d'un processus de 256 Mio prend des millisecondes: il ne copie que les tables des pages.

C tds/td3/lifecycle.c

```

/*
 * TD3 - exploration 4: the end of a process: zombies and orphans (original
 * slides 80-82 and 88).
 * 1. A child that exits stays as a ZOMBIE (state Z, <defunct>) until its
 * parent calls wait: the
 *     kernel keeps its PID and exit status. ps shows it before and after
 * waitpid.
 * 2. A process whose parent dies is an ORPHAN: the kernel gives it a new

```

```

parent, init (PID 1) or the
* nearest ancestor marked as "subreaper" (on a desktop, usually systemd
--user).
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static void pause_ms(long ms)
{
    struct timespec ts = { ms / 1000, (ms % 1000) * 1000000L };
    while (nanosleep(&ts, &ts) == -1)
        ;
}

static void ps(pid_t pid)
{
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm -p %d | sed 's/^/'", (int)pid);
    fflush(stdout);
    if (system(cmd) == -1)
        perror("system");
}

/* Name of a process, from /proc/PID/comm. */
static void comm_of(pid_t pid, char *name, size_t len)
{
    char path[64];
    snprintf(path, sizeof path, "/proc/%d/comm", (int)pid);
    FILE *f = fopen(path, "r");
    if (f == NULL || fgets(name, (int)len, f) == NULL)
        snprintf(name, len, "?");
    name[strcspn(name, "\n")] = '\0';
    if (f != NULL)
        fclose(f);
}

int main(void)
{

```

```

/* 1. Zombie */
pid_t child = fork();
exit_if(child == -1, "fork");
if (child == 0)
    _exit(7);
pause_ms(200);
printf("1) the child %d has exited, but nobody has called wait yet:\n",
(int)child);
ps(child);
int status;
exit_if(waitpid(child, &status, 0) == -1, "waitpid");
printf("    waitpid → WIFEXITED = %d, WEXITSTATUS = %d; now ps finds
nothing:\n", WIFEXITED(status), WEXITSTATUS(status));
ps(child);

/* 2. Orphan: the middle process exits, the grandchild is adopted */
printf("2) orphan: my PID is %d\n", (int)getpid());
fflush(stdout);
pid_t middle = fork();
exit_if(middle == -1, "fork");
if (middle == 0) {
    pid_t grandchild = fork();
    exit_if(grandchild == -1, "fork");
    if (grandchild == 0) {
        pid_t before = getppid();
        pause_ms(200);                                /* meanwhile its parent
exits */
        pid_t after = getppid();
        char name[64];
        comm_of(after, name, sizeof name);
        printf("    grandchild %d: parent was %d, now it is %d (%s)\n",
(int)getpid(), (int)before, (int)after, name);
        exit(EXIT_SUCCESS);
    }
    exit(EXIT_SUCCESS);                                /* the grandchild becomes
an orphan */
}
exit_if(waitpid(middle, NULL, 0) == -1, "waitpid");
pause_ms(400);                                        /* let the grandchild
report */
return EXIT_SUCCESS;
}

```

Sortie réelle

```
$ ./lifecycle
```

1) the child 356667 has exited, but nobody has called wait yet:

```
PID    PPID STAT COMMAND
356667 356666 Z    lifecycle
```

waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:

```
PID    PPID STAT COMMAND
```

2) orphan: my PID is 356666

grandchild 356679: parent was 9771, now it is 9771 (systemd)

L'enfant terminé affiche l'état **Z** jusqu'au **waitpid**, qui récupère son code (7). Le petit-enfant orphelin ne va pas à **init** mais au **systemd** de la session utilisateur, un *subreaper* (`prctl(PR_SET_CHILD_SUBREAPER)`).

Le voir animé → A4.1 mémoire virtuelle · A4.2 copie sur écriture · A4.3 états

Liste de vérification

Défi avec l'IA générative · Un mini-shell



Objectif. Transformez `launch` en un interpréteur de commandes minimal: il lit des lignes, exécute les commandes avec `fork`/`execvp`, gère `>`; fichier et affiche le statut de fin.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

À partir de mon `launch.c` [le coller], aide-moi à écrire un mini-shell en C qui :
lit des lignes avec `getline`, les découpe en mots, exécute la commande avec `fork + execvp`, gère la redirection `>` fichier avec `dup2`, implémente `cd` et `exit` comme commandes internes (explique pourquoi `cd` ne peut pas être un programme externe) et affiche `[status N]` ou `[signal SIGxxx]` quand une commande se termine.
Demande-moi d'écrire chaque partie avant de me montrer la tienne.

Comment vérifier le résultat

- `ls -l > out.txt` crée le fichier et le shell continue de fonctionner.
- `cd /tmp` suivi de `pwd` affiche `/tmp`.
- Lancer `./segfault` affiche le signal et ne laisse aucun processus zombie (`ps`).

Déclarez votre usage de l'IA: outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.